

Blumetara AI: Comprehensive Technical Documentation

This document serves as the primary technical reference for the **Blumetara AI** codebase. It outlines the system architecture, directory configurations, database schemas, core backend services (RAG, OCR, AI), mobile/admin client layouts, and deployment pipelines.

1. System Architecture & Component Interactions

Blumetara AI leverages a decoupled architecture designed to scale seamlessly up to 10,000+ users. The core database operations and vector indexes are consolidated in **MongoDB Atlas**, while compute-heavy tasks like document OCR parsing and RAG chunking are handled asynchronously.



2. Directory Structure Map

The monorepo workspace at `/Users/pranav\_ns/Desktop/Blumetara.ai` contains three main packages:

```
Blumetara.ai/├── backend/                                # FastAPI Application | ─── app
```

3. Database Schema Models (MongoDB Atlas)

To maintain schema flexibility and enable RAG context vector matches in a single stack, the system is designed around **MongoDB Atlas**.

3.1 `USERS` & `PROFILES`

Tracks user credentials, roles, and demographic markers required for AI context personalization.

```
// users{  "_id": "ObjectId",  "firebaseUid": "string (unique index)",  "e
```

3.2 `HEALTH\_REPORTS` & `REPORT\_CHUNKS`

Stores uploaded PDF file references, parsed text content, and vector arrays for RAG.

```
// health_reports{  "_id": "ObjectId",  "userId": "string",  "s3Key": "st
```

3.3 `MEDICINES` & `REMINDERS`

Powers adherence calendars and pill counts. Marking a reminder logs updates and decrements stock.

```
// medicines{  "_id": "ObjectId",  "userId": "string",  "name": "string",
```

4. Backend APIs & Endpoint Registry

All routes are prefix-scoped with `/api/v1` and validation is enforced via Pydantic schemas:

Category	HTTP Method	Endpoint	Description	Auth Required
Auth	`GET`	`/auth/me`	Fetches Firebase user credentials & roles	Yes
Profile	`GET`	`/users/profile`	Fetches active demographic metrics	Yes
Profile	`PUT`	`/users/profile`	Updates weight, age, or preferences	Yes
Reports	`POST`	`/reports/upload`	Uploads PDF to S3, returns metadata ID	Yes
Reports	`GET`	`/reports`	Lists user documents with presigned view links	Yes
Reports	`POST`	`/reports/{id}/analyze`	Runs Textract OCR, chunks & embeds content	Yes
AI Chat	`POST`	`/ai/chat`	Chat with TARA (RAG & safety intercept enabled)	Yes
Goals	`POST`	`/goals`	Instantiates step, hydration, or sleep goals	Yes
Reminders	`POST`	`/reminders`	Configures pill alarms and drinking intervals	Yes
Admin	`GET`	`/admin/telemetry`	Admin-only metrics dashboard feed	Yes (Admin)

5. TARA AI Agent: Parallel-Isolated Reasoning &

Consensus

TARA uses a feature flag toggle ``AI_REASONING_MODE`` inside ``config.py`` to support two distinct routing modes:

5.1 LITE SINGLE-AGENT MODE (``AI_REASONING_MODE="LITE"``)

Executes a single standard, fast, and token-efficient call to Gemini Flash. The model is guided by system instructions outlining safety compliance (do not diagnose, check safety boundaries).

5.2 ENTERPRISE MULTI-AGENT CONSENSUS MODE (``AI_REASONING_MODE="ENTERPRISE"``)

Rather than risking cognitive bias or "groupthink" in a single conversation history, the system dispatches three expert agents in **Parallel Isolated Tasks** (``asyncio.gather``):

1. **Clinical Lab Analyst**: Focuses strictly on lab ranges, vitals, and reference bounds.
2. **Lifestyle & Wellness Coach**: Focuses on patient habits, hydration targets, and physical movement.
3. **Medical Safety & Adherence Officer**: Scans prescription schedules, overlaps, and safety flags.

Once the isolated reports are completed, the **Consensus Aggregator (Chief Medical Synthesizer)** compiles the findings, removes conflicting reports, formats the text, and attaches mandatory medical disclaimers.

5.3 SAFETY GUARDRAILS (ACTIVE IN BOTH MODES)

- **Emergency Intercept**: Scans query streams for emergency terms (chest pain, breathing difficulty, etc.) and aborts the LLM connection, returning an immediate warning directive to seek professional care.

- **Off-Topic Filter:** Rejects prompts unrelated to health, medicine, biology, or exercise, protecting system token use.
-

6. Local Sandbox & Mocks (Zero-Dependency Testing)

To enable rapid offline testing before cloud accounts and credentials are configured, the codebase runs in **mock fallback mode** whenever credentials are empty:

- **Firebase JWT:** Decodes local developmental mock tokens (``mock_user_123_admin``) for user mapping.
 - **AWS S3:** Bypasses network calls and writes raw bytes locally to ``backend/local_s3_mock/``.
 - **AWS Textract:** Returns structured mock laboratory output matching realistic blood test parameters.
 - **Vector Search:** Performs local CPU-based cosine-similarity matches using ``numpy`` if Atlas vector indexes are unavailable.
 - **Gemini AI:** Generates custom rule-based responses if the API key is empty.
-

7. Automated Verification Pipeline

All backend services are validated using an automated verifier in ``backend/test_api.py``. Running ``backend/.venv/bin/python backend/test_api.py`` checks:

- **Test 1:** Decoding mock Firebase token and verifying role extraction.

- **Test 2:** Uploading content to S3 and verifying mock directory output.
- **Test 2.1:** Mock OCR parsing through AWS Textract.
- **Test 3:** Document text chunking, embedding generation, and RAG local cosine-similarity matches.
- **Test 4:** Grounded chat responses containing demographic data and context references.
- **Test 5:** Safety interrupts blocking severe cardiac/respiratory queries.
- **Test 6:** Prompt validation rejecting off-topic code/history requests.

All six tests pass successfully in both LITE and ENTERPRISE modes.